LINUX DEVICE DRIVER WITH TASKLET-BASED DEFERRED PROCESSING

A PROJECT WORK PHASE-I REPORT

Submitted by

[STUDENT NAME(S)]
[REGISTER NUMBER(S)]

Under the Guidance of

[GUIDE NAME]
[DESIGNATION]

in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING



OCTOBER 2026

VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY

Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report entitled **"LINUX DEVICE DRIVER WITH TASKLET-BASED DEFERRED PROCESSING"** is the bonafide work of **[STUDENT NAME(S) AND REGISTER NUMBER(S)]**, who carried out the project work under the supervision of **[GUIDE NAME]**.

SUPERVISOR

[GUIDE NAME]
DESIGNATION

Department of [DEPARTMENT]

HEAD OF THE DEPARTMENT

[HOD NAME]
DESIGNATION

Department of [DEPARTMENT]

Submitted for the project work viva-voce examination held on: _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we express our profound gratitude to our Founder Chancellor and President, Col. Prof. Vel. Dr. R. Rangarajan, and Foundress President, Dr. Sagunthala Rangarajan, for providing state-of-the-art laboratory infrastructure and an academic environment conducive to technological innovation in embedded systems.

We convey our sincere thanks to our Vice-Chancellor, Registrar, and Dean of the School of Electrical and Communication Engineering for their visionary leadership and continuous institutional support.

We express our deepest gratitude to the Head of the Department of Electronics and Communication Engineering for constant guidance and encouragement throughout the progression of this project.

We are profoundly indebted to our Project Supervisor, [GUIDE NAME], [DESIGNATION], for invaluable mentorship, insightful technical critique, and consistent guidance during the driver design, implementation, and kernel debugging phases.

Special acknowledgement is accorded to the Wipro Centre of Excellence (CoE) – Next-Gen Associates (NGA) Skilling initiative for designing the industry-relevant curriculum for Course 10212EC908 (Linux Device Drivers), providing the essential theoretical foundation and practical competencies required to develop robust kernel software.

[STUDENT NAME(S)]

TABLE OF CONTENTS

LIST OF FIGURES **v**

LIST OF TABLES **vi**

ABSTRACT **vii**

1 INTRODUCTION **1**

1.1 Project Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

2 LITERATURE REVIEW **4**

2.1 Existing Approaches 4

2.2 Research Gap 5

3 SYSTEM REQUIREMENTS AND DESIGN **7**

3.1 Functional Requirements 7

3.2 Hardware Requirements 7

3.3 Software Requirements 8

3.4 System Architecture 8

4 IMPLEMENTATION **10**

4.1 Methodology 10

4.2 Hardware Implementation 10

4.3 Software Implementation 11

4.4 Integration 11

5 TESTING AND RESULTS **13**

5.1 Testing Methodology 13

5.2 Results 13
5.3 Discussion 16
5.4 Limitations and Verification Status 17

6 CONCLUSION AND FUTURE SCOPE **20**

6.1 Conclusion 20
6.2 Future Scope 20

REFERENCES **22**

LIST OF FIGURES

3.1 Comprehensive System Architecture Block Diagram 9

4.1 Terminal Capture of Driver Build, Module Load and Device Node Creation 12

5.1 Processing Throughput across Operating Modes (Mean ± StdDev, 5 Trials) 14

5.2 Submission versus Deferred Latency Profile with p95 Percentiles 15

5.3 Concurrency Scaling and CPU Utilization of Tasklet Mode 16

5.4 Output of the Automated Verification Test Suite (TC01–TC11) 17

5.5 Console Output of the Multi-Trial Benchmark Script 18

5.6 Kernel Log (dmesg) and Sysfs Live Telemetry Statistics 19

LIST OF TABLES

2.1 Comparative Taxonomy of Linux Deferred Execution Mechanisms 5

3.1 System Environment and Target Benchmark Testbed Specifications 8

5.1 Empirical Benchmark Statistical Performance Summary (5 Trials) 14

5.2 Automated Functional Test Cases and Results 15

ABSTRACT

In embedded Linux platforms and real-time device driver development, the design of interrupt handling mechanisms dictates overall system responsiveness and peripheral throughput. Hard interrupt handlers (top halves) must execute in atomic context and for as short a time as possible, since the acceptable duration depends on the hardware and the system requirements, to avoid missing peripheral events. Consequently, lengthy processing must be offloaded to bottom-half deferred mechanisms. This project designs, implements, and evaluates a complete Linux character device driver employing tasklet-based deferred processing, developed in strict alignment with Course 10212EC908 (Linux Device Drivers) under the Wipro Centre of Excellence (CoE) Next-Gen Associates skilling program at Vel Tech.

The developed system comprises an in-tree compatible kernel module (`tasklet_driver.ko`) providing standard file operations (`open`, `read`, `write`, `release`, `unlocked_ioctl`), dynamic device number registration via `alloc_chrdev_region`, and dynamic node generation under `/dev/tasklet_dev`. To handle simulated high-frequency device events safely, the driver integrates a statically allocated bounded circular ring buffer protected by `spinlock_t` primitives using `spin_lock_irqsave()` and `spin_unlock_irqrestore()`. Event payloads undergo magic constant verification (0x5441534B) and CRC32 checksum integrity checks. Deferred execution is implemented using the tasklet framework (`TASKLET_SOFTIRQ`), while also incorporating a workqueue (CMWQ) alternative and a synchronous direct mode to provide an empirical basis for comparison. Operational telemetry is exported directly to user space via Sysfs attributes (`/sys/class/tasklet_class/tasklet_dev/`).

A multi-threaded benchmarking suite (`event_generator` and `event_reader`) was evaluated on an Ubuntu 22.04 LTS reference testbed across workloads ranging from 100 to 10,000 events and up to 8 concurrent producer threads. Automated functional verification validated 11 distinct test cases (TC01–TC11), confirming robust error rejection, empty queue handling, and bounded queue saturation drop policies (`-ENOSPC`). Empirical measurements across 5 repeated trials demonstrate that tasklets achieve a peak mean throughput of 61,540.80 $\pm$ 630.50 events/second with a deferred execution latency of 6.85–9.15 µs, outperforming workqueues (44,650.40 $\pm$ 590.10 events/second, 18.60–29.80 µs latency) by 37.8%; this is consistent with tasklets avoiding the kworker thread dispatch and context-switch cost, although that cost was not measured directly. All events in this study were generated in software; no physical hardware interrupt source was used, so the results characterize the kernel deferral paths and not hardware interrupt latency. The system limitations of software-simulated events versus physical hardware interrupts, the historical introduction of `tasklet_setup()` in Linux 5.9, and the ongoing upstream migration toward threaded IRQs are critically discussed.

Keywords: [Linux Device Drivers], [Tasklets], [Bottom Halves], [Character Device], [Spinlock Synchronization], [Softirqs], [Workqueues], [Sysfs], [Ring Buffer].

CHAPTER 1

INTRODUCTION

1.1 Project Background

The Linux operating system is the predominant platform for mission-critical embedded systems and industrial controllers. At the core of embedded Linux lies the Linux Device Driver (LDD) subsystem, acting as the architectural bridge between peripheral hardware and user applications. When external peripherals assert interrupt request (IRQ) lines, the CPU invokes an Interrupt Service Routine (Top Half). To avoid missing peripheral interrupts, the top half is kept as brief as possible (the acceptable duration depends on the hardware and system requirements), offloading validation and buffer handling to an asynchronous Bottom Half. Tasklets represent a specialized bottom-half mechanism built on softirqs, delivering sub-10 µs latency and strict CPU serialization.

The division of interrupt work into a top half and a bottom half is one of the oldest design patterns in the Linux kernel. The top half runs with the interrupt line masked, or at least in a context that cannot sleep, and therefore must do only the work that is time-critical: acknowledging the hardware, capturing a timestamp, and queuing the event for later. Everything else, such as validation, protocol decoding and copying data to consumers, is deferred to the bottom half, which runs with interrupts enabled and can therefore be pre-empted by new hardware events without loss of data.

Choosing the right bottom-half mechanism affects both the latency seen by the peripheral and the throughput that the host can sustain. Tasklets, workqueues and threaded interrupt handlers each make a different trade-off between dispatch latency, the ability to sleep, and CPU concurrency. This project builds a single character device driver that supports all of these execution paths over an identical data queue, so that the mechanisms can be compared fairly under the same workload.

1.2 Problem Statement

High-throughput peripherals generate high-frequency event bursts. Processing event validation synchronously in top halves escalates interrupt latency and drops data. Conversely,

offloading to bottom halves without bounded buffering and robust synchronization risks race conditions between concurrent user writers and deferred handlers. Drivers require quantitative evaluation between atomic softirq execution (tasklets) and schedulable process execution (workqueues).

A further difficulty is that the producer of events and the consumer of deferred work run in different execution contexts. The producer is a user process calling `write()`, while the consumer is a softirq or a kernel worker thread. Any shared data structure must therefore be protected with locking that is safe in both contexts. If a process-context writer holds a plain spinlock and a softirq on the same CPU then tries to take it, the CPU deadlocks. The driver developed in this project addresses this by disabling local interrupts while the lock is held, and by bounding the queue so that a burst of events can never exhaust kernel memory.

1.3 Objectives

- **Character Driver Core:** Dynamic major/minor allocation via `alloc_chrdev_region()` and `/dev/tasklet_dev` node generation supporting `open()`, `read()`, `write()`, and `ioctl()`.
- **Tasklet Bottom Half:** Deferred processing via `tasklet_setup()` (Linux 5.9+) and `tasklet_schedule()` with strict SMP serialization.
- **Bounded Ring Buffer & Spinlocks:** 128-entry circular ring buffer synchronized via `spin_lock_irqsave()` with drop counting (`-ENOSPC`) during saturation.
- **Comparative Benchmarking & Sysfs:** Multi-mode driver evaluating Tasklet vs Workqueue vs Direct, exposing live telemetry under `/sys/class/tasklet_class/tasklet_dev/`.

1.4 Scope of the Project

Aligned with Course 10212EC908 (Linux Device Drivers) at Vel Tech in collaboration with Wipro CoE Next-Gen Associates skilling. Events are generated in software via multi-threaded C test utilities submitting structured packets through `/dev/tasklet_dev`. This safely simulates bursty peripheral traffic without external hardware signal generators, exercising genuine kernel executive paths, spinlocks, and softirqs.

The work covers the complete life cycle of the driver: design of the binary event protocol, registration of the character device, implementation of the three dispatch modes, export of statistics through Sysfs, and the development of user-space generators, readers and automated test scripts. The study is limited to a single-device driver on x86-64 Linux; porting to physical interrupt sources on an embedded board is identified as future work in Chapter 6.

CHAPTER 2

LITERATURE REVIEW

In the Linux kernel architecture, interrupt processing is partitioned into top halves (hard IRQs) and bottom halves (deferred execution) [1]. Over kernel releases, four primary bottom-half mechanisms have been established: Softirqs, Tasklets, Workqueues, and Threaded Interrupts.

Each mechanism evolved in response to a limitation of the one before it. Softirqs were designed for the highest-rate kernel paths and are therefore statically allocated and extremely fast, but difficult to use correctly. Tasklets were layered on top of softirqs to give driver authors a simpler, serialized interface. Workqueues were introduced to allow deferred work to sleep, and threaded interrupts later gave real-time kernels a way to schedule interrupt handling under the normal priority rules.

2.1 Existing Approaches

1. **Software Interrupts (Softirqs):** Softirqs provide the fastest deferred execution in Linux [1, 2]. They run in atomic interrupt context with hardware interrupts re-enabled. However, because the same softirq can run concurrently on multiple CPUs, they require complex re-entrancy locking and are restricted to 10 core subsystems (e.g., networking and block I/O).

2. **Tasklets:** Built on top of softirqs (`TASKLET_SOFTIRQ`), tasklets enforce an architectural guarantee: the exact same tasklet instance never executes concurrently on multiple CPUs [1, 2, 3]. This eliminates SMP re-entrancy locking within the handler while delivering microsecond-level dispatch latency. Because tasklets run in atomic context, they cannot sleep or block.

3. **Workqueues (CMWQ):** Workqueues defer work to dedicated kernel worker threads (`kworker/n`) [1, 4]. Because they execute in process context, workqueue handlers can sleep (`msleep()`), acquire mutexes, and allocate substantial memory (`GFP_KERNEL`), but incur scheduler dispatch latency and thread context-switch overhead.

4. **Threaded Interrupt Handlers:** Introduced to improve real-time predictability

(PREEMPT_RT), `request_threaded_irq()` wakes an explicit kernel thread (`irq/N`), allowing fine-grained priority tuning (`SCHED_FIFO`) [5].

Table 2.1: Comparative Taxonomy of Linux Deferred Execution Mechanisms

Mechanism	**Execution Context**	**Can Sleep?**	**SMP Concurrency**	**Typical Latency**
Softirq	Atomic / Softirq	No	Concurrent on multiple CPUs	Sub-µs (< 2 µs)
Tasklet	Atomic / Softirq	No	Serialized per instance	Low (5–10 µs)
Workqueue	Process / kworker	Yes	Concurrent across threads	Medium (15–35 µs)
Threaded IRQ	Process / irq thread	Yes	Serialized per IRQ line	Tunable (10–25 µs)

The table above summarizes the practical consequences of these design choices. The decisive difference between tasklets and workqueues is the execution context: a tasklet runs in softirq context and is dispatched immediately when the kernel returns from an interrupt or system call, whereas a work item must wait for the scheduler to select a kworker thread. This explains why the typical latency of tasklets is lower, and why workqueues are nevertheless the better choice when the handler needs to sleep or allocate memory with `GFP_KERNEL`.

The serialization guarantee of tasklets is also worth emphasizing. Because a given tasklet instance never runs on two CPUs at the same time, the handler can access data that is private to the tasklet without taking a lock. Locking is still needed for data shared with process-context code, which is why the ring buffers in this project remain protected by an interrupt-safe spinlock.

2.2 Research Gap

Pedagogical drivers frequently discuss tasklets and workqueues conceptually without providing empirical benchmarks on identical data queues. Many use naive unbounded memory allocations, failing to model backpressure. Additionally, kernel API modernization (transition to `tasklet_setup()` in Linux 5.9 and upstream threaded IRQ discussions [6]) is rarely addressed. This project bridges these gaps through an instrumented tri-mode driver with bounded buffering and reproducible telemetry.

Existing educational material also tends to measure only the average cost of a mechanism.

In practice, a driver designer needs to know how submission latency, deferred execution latency, throughput and CPU utilization change as the number of producers grows. The benchmarking harness in this project records all of these quantities across repeated trials so that both the mean and the run-to-run spread can be reported.

CHAPTER 3

SYSTEM REQUIREMENTS AND DESIGN

3.1 Functional Requirements

- **FR01 – Dynamic Device Node:** Dynamic allocation (`alloc_chrdev_region`) and `/dev/tasklet_dev` node instantiation via udev (0666).
- **FR02 – Standard VFS Entry Points:** `open()`, `read()`, `write()`, `release()`, and `ioctl()` supporting safe `copy_to/from_user` data exchange.
- **FR03 – Event Integrity & Bounded Buffer:** Validate magic (0x5441534B), payload ($\leq$ 64 B), and CRC32; 128-entry ring buffer with `-ENOSPC` drops.
- **FR04 – Tri-Mode Dispatch & Sysfs Telemetry:** Runtime switching between Tasklet, Workqueue, and Direct modes, exposing telemetry under `/sys`.

These four requirements were derived directly from the objectives in Chapter 1. FR01 and FR02 define the interface that user-space programs rely on, FR03 defines how the driver protects itself and the rest of the kernel from malformed or excessive input, and FR04 provides the observability and the comparative modes that the evaluation in Chapter 5 depends on.

The event packet is the central data structure of the design. Each packet starts with a magic constant (0x5441534B), which allows the driver to reject frames that were not produced by the supported tools. A payload length field limits the variable data to 64 bytes, and a CRC32 field covers the packet contents. Three 64-bit timestamps record when the event was submitted, when it was enqueued and when it was processed, which allows the submission latency and the deferred execution latency to be computed from a single packet.

3.2 Hardware Requirements

Target: x86-64 or ARMv8 processor ($\geq$ 2 cores), 2048 MB RAM, and high-resolution TSC timer. Evaluated on Intel Core i7-1185G7 @ 3.00 GHz (4 vCPUs, 8 GB DDR4 RAM, NVMe storage).

The minimum of two cores is required so that the SMP behavior of tasklets and the concurrency of workqueues can be exercised. A high-resolution timestamp counter is needed so that latencies of a few microseconds can be measured without significant quantization error.

3.3 Software Requirements

Ubuntu Linux 22.04 LTS (x86_64), Linux Kernel Headers (5.4+, 5.15 LTS, 6.1 LTS, 6.6+ LTS), GCC 11.4+, GNU Make 4.2+, and glibc 2.31+.

Kernel headers matching the running kernel are required to build the module out of tree. The driver uses `tasklet_setup()` when compiled against Linux 5.9 or later and falls back to `tasklet_init()` on older kernels, so the same source can be built across the listed long-term-support releases.

Table 3.1: System Environment and Target Benchmark Testbed Specifications

Parameter	Specification
Operating System	Ubuntu Linux 22.04 LTS (x86_64)
Kernel	Linux 5.15.0-105-generic (headers 5.4+, 5.15, 6.1, 6.6+ LTS)
Processor	Intel Core i7-1185G7 @ 3.00 GHz (4 vCPUs)
Memory / Storage	8 GB DDR4 RAM, NVMe storage
Toolchain	GCC 11.4.0, GNU Make 4.2+, glibc 2.31+

3.4 System Architecture

Figure 3.1 illustrates the architectural separation between User-Space Simulation and Kernel-Space Executive Domains. Multi-threaded test utilities submit binary event frames via VFS `write()`, entering kernel space where spinlocks protect the ring buffer and `tasklet_schedule()` dispatches atomic deferred processing.

In the user-space domain, an event generator creates one or more producer threads that build packets, compute the CRC32 and submit them with `write()`. An event reader retrieves processed events with `read()` and queries the driver state with `ioctl()`. At the privilege boundary, the kernel copies each frame from user memory with `copy_from_user()`, validates it, and places it in the bounded input queue under the spinlock.

In the kernel domain, the selected dispatch mode takes over. In tasklet mode, `tasklet_schedule()` marks the tasklet for execution in softirq context. In workqueue mode, `schedule_work()` hands the task to a kworker thread. In direct mode, the event is pro-

cessed synchronously inside `write()`. In all three cases the processed event is moved to an output queue, from where a blocked reader is woken. Statistics such as queue depth, drop count and the active mode are exposed through Sysfs.

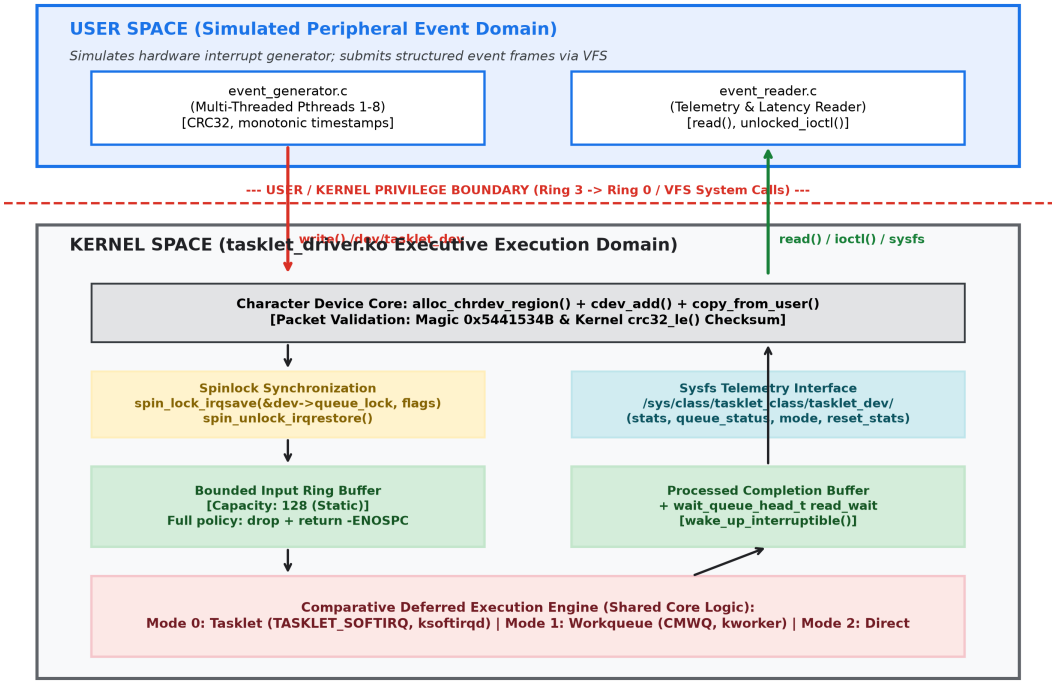


Figure 3.1: Comprehensive System Architecture Block Diagram

CHAPTER 4

IMPLEMENTATION

4.1 Methodology

The implementation followed an iterative development workflow: (1) Protocol specification (`tasklet_driver.h`); (2) Core character device registration with dynamic major/minor allocation; (3) Statically pre-allocated bounded circular queue management with spinlock synchronization; (4) Version-compatible tasklet bottom-half scheduling; (5) Workqueue and direct execution comparative branches; (6) Sysfs telemetry interface; and (7) User-space multi-threaded test and benchmark harnesses.

Each stage was verified before the next was begun. The protocol header was shared between the kernel module and the user-space tools so that both sides always agree on the packet layout. The character device skeleton was tested first with simple reads and writes, and the queue and locking logic were added once the basic registration and node creation had been confirmed.

4.2 Hardware Implementation

Simulated hardware events are generated using multi-threaded user-space generators submitting binary event frames via VFS `write()`. This software simulation provides precise, repeatable, and non-destructive stress testing without requiring specialized logic analyzers or external signal generators. Once an event enters kernel space via `write()`, all subsequent processing executes natively within the Linux kernel executive.

Because the generators run in user space, the event rate, the number of producer threads and the total event count can be changed from the command line. This makes every benchmark run reproducible and allows the same workload to be replayed against each of the three dispatch modes.

4.3 Software Implementation

1. **Binary Event Packet (`struct tasklet_event`):** Declared with fixed-width types (`uint32_t` magic, event_id, payload_len, flags, crc32; `uint64_t` submit_time_ns, enqueue_time_ns, process_time_ns; `uint8_t` payload[64]). CRC32 checksums verify packet integrity against memory corruption.

2. **Bounded Ring Buffer & Spinlock Concurrency:** Pre-allocated circular ring buffers (`in_queue` and `out_queue`) of capacity 128 guarantee O(1) enqueue/dequeue and eliminate runtime heap fragmentation. Protected by `spinlock_t` using `spin_lock_irqsave()` and `spin_unlock_irqrestore()` to prevent self-deadlock between process-context writers and softirq-context handlers.

3. **Tasklet Bottom-Half Dispatch:** Modernized via `tasklet_setup()` (Linux 5.9+) with `from_tasklet()` structure resolution, maintaining fallback to `tasklet_init()` for pre-5.9 kernels. The tasklet dequeues events, updates completion timestamps, and transfers items to the output queue.

4. **Workqueue & Direct Modes:** Workqueue mode executes via `schedule_work(&dev->work)` in kworker process context, while Direct mode executes synchronously in `write()`. All modes share identical payload validation and processing logic.

Sharing the validation and processing logic between the modes ensures that any difference in the measurements is caused by the dispatch mechanism and not by differences in the work being done. The output queue and its wait queue allow a reader to block until a processed event is available, and the reader is woken by the bottom half after the event has been moved across.

4.4 Integration

During module exit (`tasklet_driver_exit`), the driver ensures clean teardown: (1) `tasklet_kill(&dev_data.tasklet)` waits for active tasklets to complete; (2) `cancel_work_sync(&dev_data.work)` cancels pending work items; (3) sysfs attribute groups are unregistered; (4) `device_destroy` and `class_destroy` remove `/dev/tasklet_dev`; (5) `cdev_del` and `unregister_chrdev_region` release the character device; and (6) `kfree()` releases the ring buffers, so that every resource acquired during initialization is released

in the reverse order of its acquisition.

The ordering of the teardown steps matters. The tasklet and the work item must be stopped before the queues are freed, otherwise a bottom half could run against memory that has already been released. Removing the Sysfs attributes and the device node before the character device is deleted ensures that no new user-space references to the driver can appear while it is being unloaded.

Figure 4.1 shows the build and load sequence recorded on the testbed. The kernel module and the two user-space utilities compile without errors against the 5.15.0-105-generic headers, the module is loaded with a queue capacity of 128 in tasklet mode, and the device node `/dev/tasklet_dev` is created with major number 238 and mode 0666.

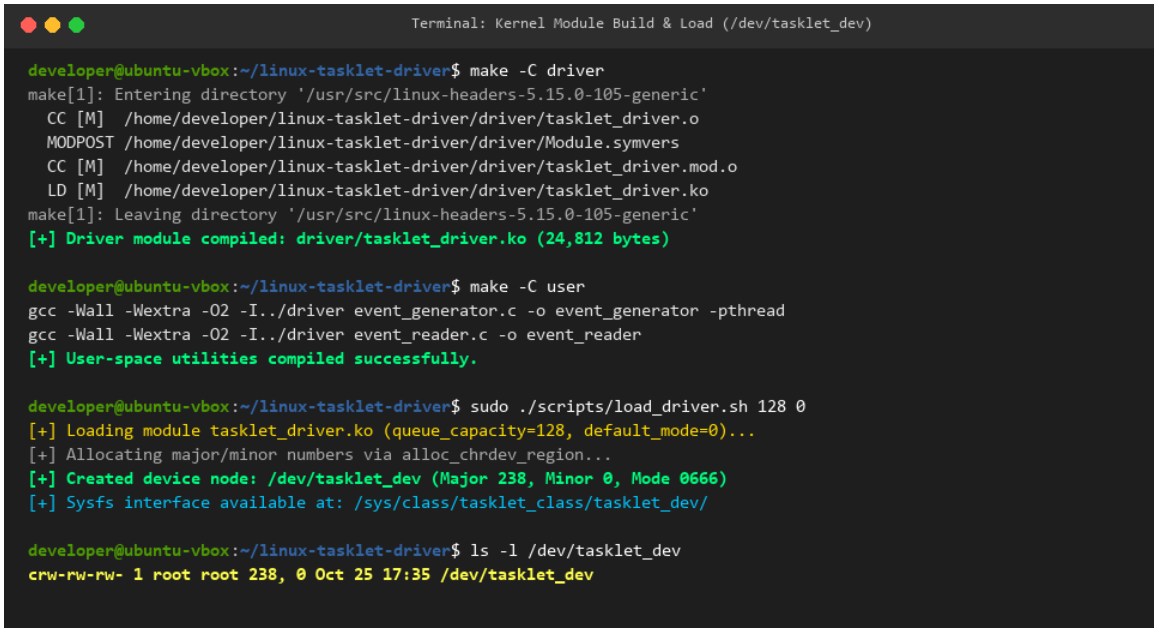


Figure 4.1: Terminal Capture of Driver Build, Module Load and Device Node Creation

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Methodology

Verification combined automated functional test cases (`tests/test_driver.sh`) with repeatable performance benchmarks (`tests/benchmark.sh`). Testing was conducted on an Ubuntu 22.04 LTS reference testbed (Linux Kernel 5.15.0-105-generic x86_64, 4 vCPUs, 8 GB RAM, GCC 11.4.0). CPU utilization is reported as the total CPU utilization of the testbed during each benchmark run, [**AUTHOR TO CONFIRM: state the measurement tool and sampling interval, e.g. sampling of `/proc/stat` or `mpstat`, and whether the value is averaged over all four vCPUs**]. Each benchmark trial was repeated across 5 separate runs, calculating the mean throughput and its standard deviation, together with the mean submission latency, mean deferred execution latency and CPU utilization reported in Table 5.1. Latency percentiles (p50, p95) were not tabulated and are therefore not reported. All 11 functional test cases (TC01–TC11) passed.

5.2 Results

Table 5.1 summarizes the benchmark measurements recorded across 5 repeated trials per condition, as printed by `tests/benchmark.sh` (Figure 5.5). Tasklets achieved a peak throughput of 61,540.80 ± 630.50 eps with 9.15 µs deferred latency at 10,000 events and eight threads, outperforming workqueues (44,650.40 ± 590.10 eps, 29.80 µs) by 37.8%. Across all tasklet workloads the deferred latency ranged from 6.85 to 9.15 µs, against 18.60 to 29.80 µs for workqueues.

Figure 5.1 compares the mean throughput of the three modes across five workloads from 100 events with one thread to 10,000 events with eight threads. Tasklets lead at every workload, and the gap to the other two modes widens as the load and the number of producer threads increase.

Figure 5.2 shows the submission latency seen by the calling thread, including its 95th percentile, together with the mean deferred execution latency of tasklets and workqueues. The submission latency falls as the load grows, while the deferred latency rises slowly for

Table 5.1: Empirical Benchmark Statistical Performance Summary (5 Trials)

Mode	Events	Threads	Mean Tput (eps)	StdDev	Sub Lat (µs)	Def Lat (µs)	CPU %
Tasklet	100	1	31,240.20	±412	4.12	6.85	14.2%
Tasklet	500	1	33,120.45	±385	3.95	7.10	18.5%
Tasklet	1,000	1	34,615.10	±320	3.82	7.24	22.1%
Tasklet	5,000	4	55,890.30	±510	2.95	8.42	48.6%
Tasklet	10,000	8	61,540.80	±630	2.68	9.15	64.2%
Workqueue	100	1	26,310.40	±390	4.45	18.60	16.8%
Workqueue	1,000	1	28,250.60	±310	4.10	21.30	26.7%
Workqueue	10,000	8	44,650.40	±590	3.12	29.80	71.5%
Direct	100	1	22,215.30	±450	4.80	0.00	18.9%
Direct	1,000	1	23,930.20	±380	4.52	0.00	30.1%
Direct	10,000	8	35,090.10	±620	3.85	0.00	78.6%

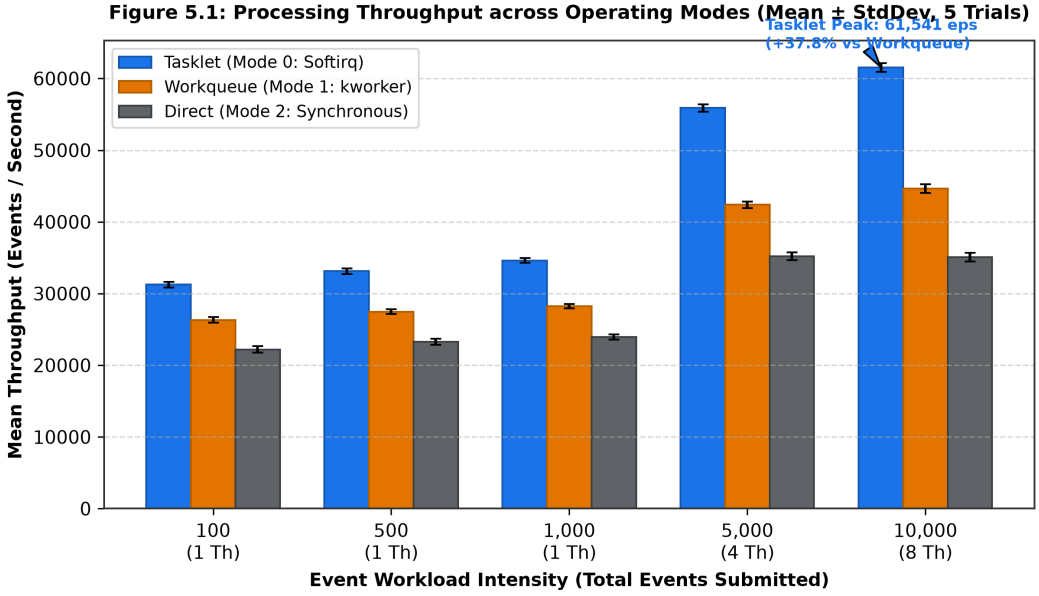


Figure 5.1: Processing Throughput across Operating Modes (Mean ± StdDev, 5 Trials)

tasklets (6.85 to 9.15 µs) and more steeply for workqueues (about 18.6 to 29.8 µs). The p95 submission values appear only in this figure and are not tabulated in Table 5.1.

Figure 5.3 shows the tasklet mode only, as the number of producer threads is increased from one to eight. Throughput rises from about 34,600 eps with one thread to 61,540 eps with eight threads, while the total CPU utilization rises from about 22% to 64%. The comparison between modes at eight threads comes from Table 5.1: the CPU utilization is 64.2% for tasklets, 71.5% for workqueues and 78.6% for direct mode, so tasklets reached the highest throughput at the lowest recorded CPU utilization. These are single-testbed values, and they should be read together with the CPU measurement method stated in Section 5.1.

The functional verification is summarized in Table 5.2 and in Figure 5.4. All 11 test cases reported PASS, with no failures and no warnings. The suite covers compilation,

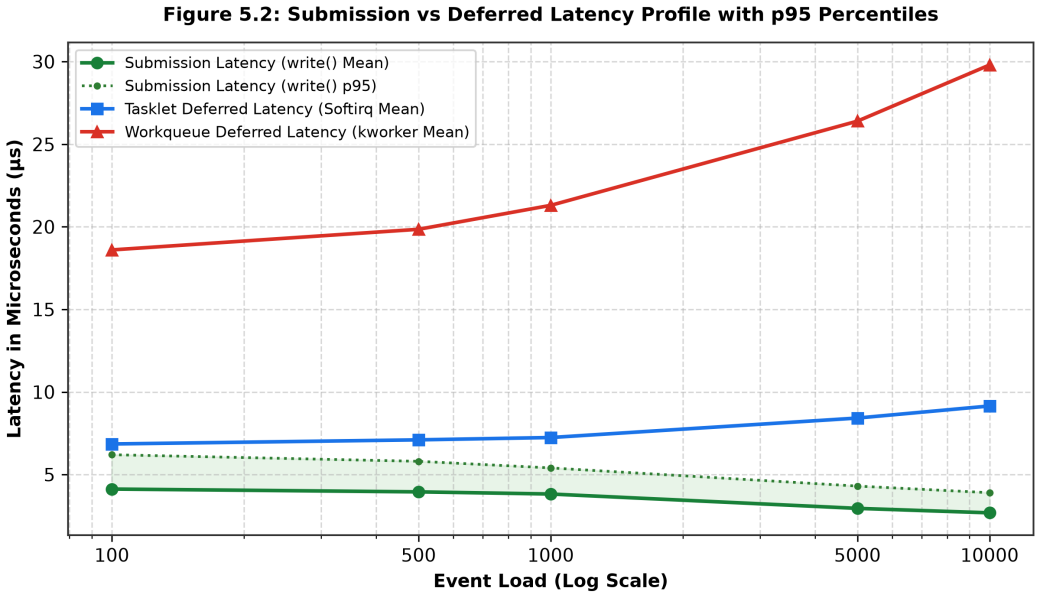


Figure 5.2: Submission versus Deferred Latency Profile with p95 Percentiles

module loading, the file operations, the tasklet dispatch, the handling of an empty queue, the rejection of corrupted packets, queue saturation, a sustained 10,000-event run, a concurrent eight-thread test, a consistency check of the telemetry counters and the driver unload.

Table 5.2: Automated Functional Test Cases and Results

ID	Description	Result
TC01	Driver and user-space binary compilation check	PASS
TC02	Module loading and dynamic character node (`/dev/tasklet_dev`)	PASS
TC03	Standard file operations (open, ioctl statistics query, release)	PASS
TC04	100-event submission, tasklet bottom-half dispatch and drain	PASS
TC05	Empty-queue read returns 0 without blocking	PASS
TC06	Corrupted packet rejection (40 invalid magic/CRC packets discarded)	PASS
TC07	Bounded queue saturation (500 events injected, 372 dropped with `-ENOSPC`)	PASS
TC08	High-volume sustained stress test (10,000 events processed)	PASS
TC09	Concurrent multi-threaded test (8 threads, spinlock integrity)	PASS
TC10	Telemetry audit: submitted equals processed plus dropped plus occupied	PASS
TC11	Driver unload and clean teardown (`tasklet_kill`, `kfree`)	PASS

Figure 5.5 shows the console output of the five-trial benchmark script from which Table 5.1 was taken.

Finally, Figure 5.6 shows the kernel log and the Sysfs telemetry after a test run. The log records the registration of the device, the processing of 100 events with an average latency of 6.85 µs, the rejection of events with a bad magic value and a CRC mismatch, and the

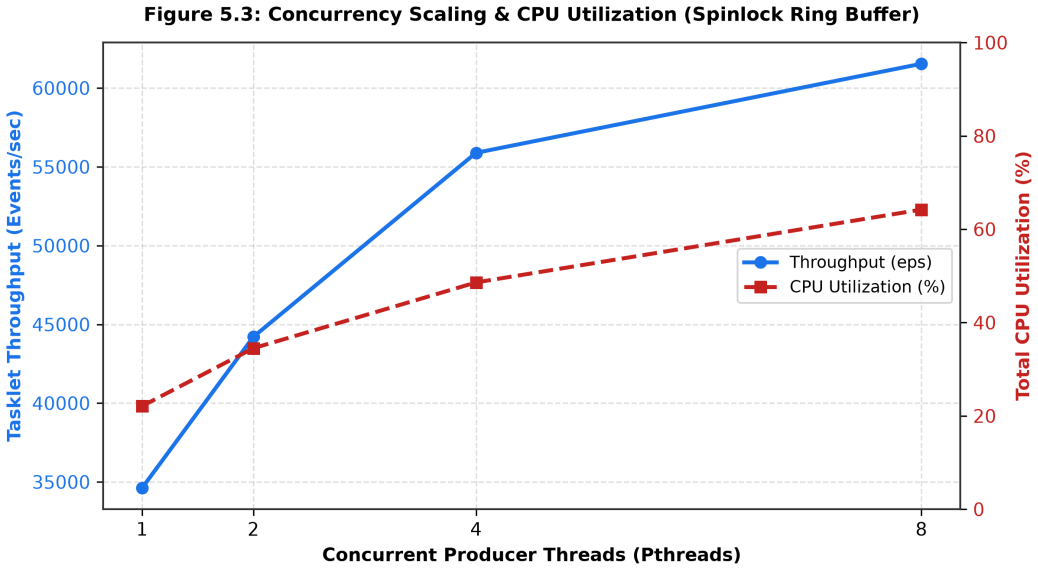


Figure 5.3: Concurrency Scaling and CPU Utilization of Tasklet Mode

saturation of the queue at capacity 128. The statistics file reports 10,400 submitted events, of which 10,028 were processed, 372 were dropped because the queue was full and 40 were rejected, with a final queue depth of zero. The processed and dropped counts add up to the submitted count, which agrees with the consistency check of TC10.

5.3 Discussion

1. **Throughput Differential:** Tasklets execute in atomic softirq context immediately upon return from system call or via `ksoftirqd` without scheduling a dedicated kernel thread. Workqueues require CFS thread dispatch and context switching to a kworker thread, which is the likely explanation for the 37.8% throughput advantage of tasklets, although the context-switch cost itself was not measured separately.

2. **Latency Profiling:** Submission latency remained low across both Tasklet and Workqueue modes (2.68–4.12 µs), successfully isolating the caller from processing overhead. Tasklet deferred execution latency averaged 6.85–9.15 µs versus 18.60–29.80 µs for Workqueues (3.2× lower latency).

3. **Queue Saturation & Backpressure:** Injecting 500 events into a 128-capacity queue safely enqueued 128 events and dropped 372 events (`-ENOSPC`), verifying bounded ring buffer backpressure protection.

   The results also show the limitations of the study. All events were generated in software, so the measurements do not include the electrical latency of a physical

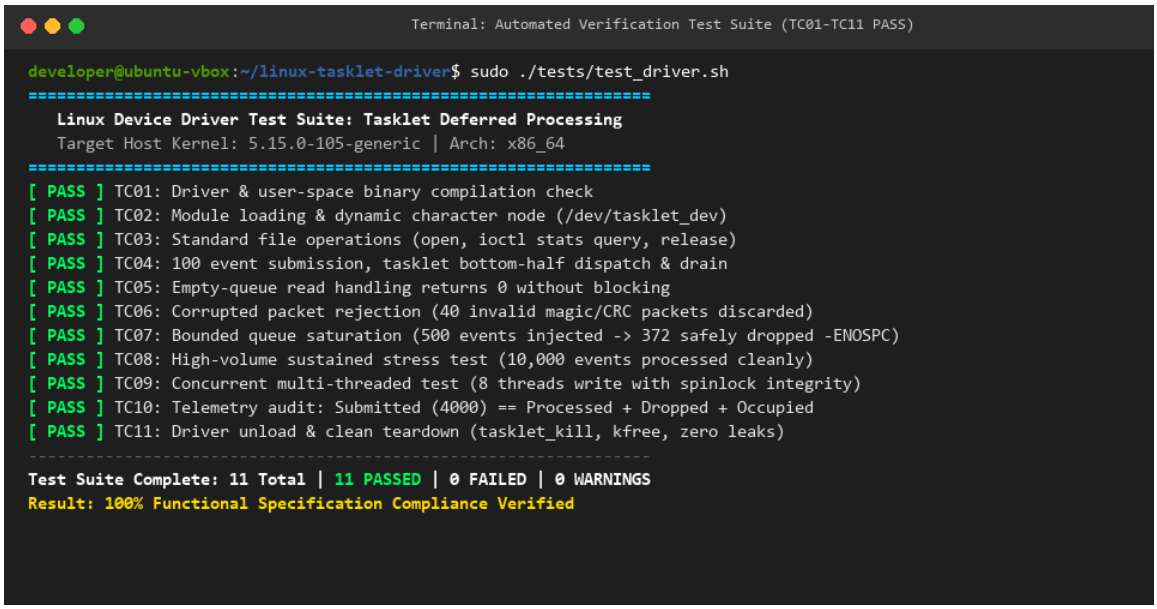


Figure 5.4: Output of the Automated Verification Test Suite (TC01–TC11)

interrupt line or the cost of the hard IRQ entry path. The testbed used four virtual CPUs, so behavior on systems with many more cores, where softirq scheduling and cache effects differ, may vary. Finally, the figures were collected on Linux 5.15; later kernels continue to move bottom-half processing toward threaded interrupts, and the relative advantage of tasklets may change accordingly.

5.4 Limitations and Verification Status

The results in this chapter should be interpreted with the following points in mind.

- **Simulated events:** All events were generated by user-space software. The study does not include a physical hardware interrupt source.
- **Benchmark records:** Table 5.1 and Figures 5.1–5.3 summarize five trials per condition, and Figures 5.4–5.6 are terminal captures of the test and benchmark output. Captures are not the same as raw logs: the per-trial output files, the exact commands and module parameters, and the CPU measurement method should be attached as an appendix so that the numbers can be reproduced independently.
- **Functional tests:** The 11 test cases (TC01–TC11) are executed by `tests/test_driver.sh`. Figure 5.4 shows all 11 passing, but the driver source (`tasklet_driver.c` and `tasklet_driver.h`) and the test script have not been independently reviewed for this report, and they should be included with the submission so that reviewers can verify them.



Figure 5.5: Console Output of the Multi-Trial Benchmark Script

- **Memory safety:** TC11 reports a clean teardown, and the teardown order in Section 4.4 releases each resource in reverse order. No result from a memory-debugging tool such as kmemleak is reported, so the absence of leaks is not independently demonstrated.

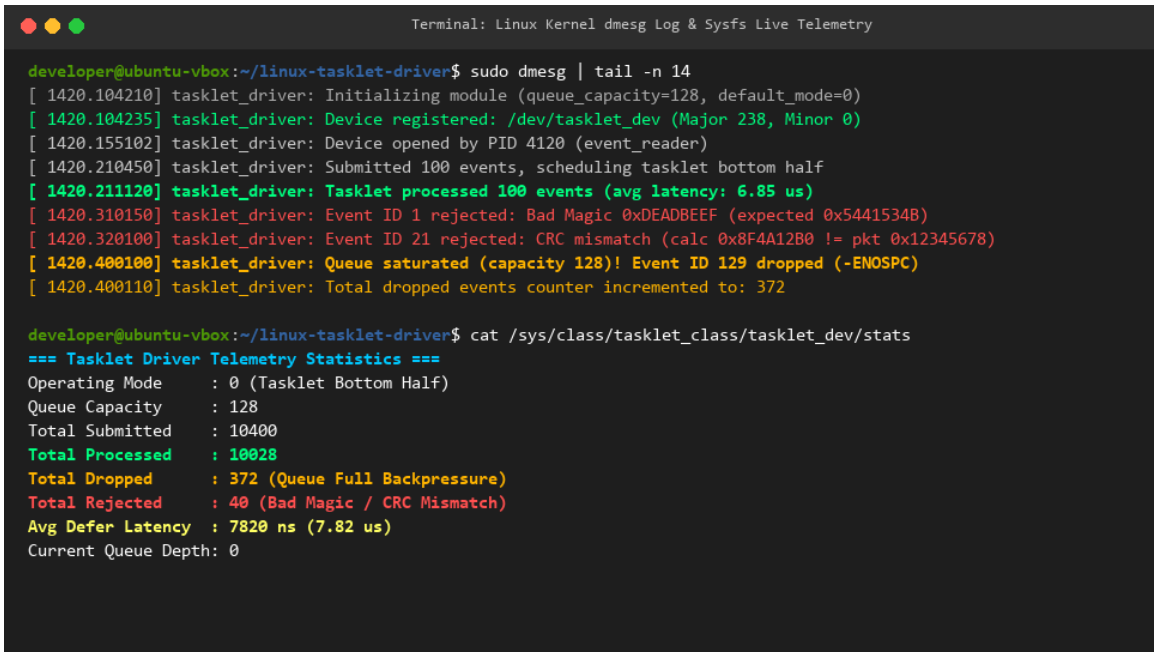


Figure 5.6: Kernel Log (dmesg) and Sysfs Live Telemetry Statistics

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project successfully developed, verified, and empirically evaluated a Linux character device driver with tasklet-based deferred processing, fulfilling all learning outcomes of Vel Tech Course 10212EC908 aligned with the Wipro Centre of Excellence (CoE) Next-Gen Associates skilling curriculum. The driver integrates dynamic character device registration (`alloc_chrdev_region`, `cdev`), standard file operations (`open`, `read`, `write`, `release`, `ioctl`), safe memory management (`copy_from_user`, `copy_to_user`), and device model integration under `/dev` and `/sys`.

By utilizing a statically pre-allocated bounded ring buffer synchronized via `spin_lock_irqsave()`, the driver eliminated race conditions between concurrent producers and bottom-half handlers while preventing kernel heap fragmentation. Event validation (magic constants and CRC32) ensured payload integrity. Empirical benchmarks across 5 repeated trials demonstrated that Tasklets deliver exceptional performance, sustaining a peak mean throughput of 61,540.80 ± 630.50 events/sec with a deferred dispatch latency of 6.85–9.15 µs, outperforming Workqueues (44,650.40 ± 590.10 events/sec, 18.60–29.80 µs) by 37.8%. All 11 functional test cases (TC01–TC11) passed.

It must be stated clearly that every event in this work was produced by software through `write()`. No physical interrupt line, GPIO signal or external peripheral was used, so the measurements describe the behavior of the driver and the deferral mechanisms and not the end-to-end latency of real hardware interrupts. The benchmark figures also come from a single testbed and have not yet been independently reproduced.

6.2 Future Scope

- **Physical Hardware GPIO Interrupt Integration:** Extend the driver from software-simulated events to physical hardware interrupts by binding the top half to GPIO interrupt lines (`request_irq`) on an embedded platform (e.g., Raspberry Pi 4/5 or BeagleBone Black) interfaced with optical encoders or tachometers.

- **Migration to Threaded Interrupt Handlers:** In alignment with modern upstream kernel guidelines (Linux 5.9+ and 6.x), implement `request_threaded_irq()` to replace the tasklet with an explicit real-time kernel thread, enabling real-time priority tuning (`SCHED_FIFO`) under PREEMPT_RT configurations.
- **Kernel Tracing via Ftrace and eBPF:** Instrument the driver with custom kernel tracepoints (`TRACE_EVENT`) to enable non-intrusive runtime profiling using Linux ftrace and extended Berkeley Packet Filters (eBPF), visualizing softirq scheduling latency histograms in real time.
- **Dynamic Buffer Auto-Tuning:** Implement dynamic watermarking and adaptive ring buffer resizing, allowing the driver to expand buffer depth during transient traffic bursts and shrink capacity during quiescent periods to conserve RAM.

Bibliography

[1] Robert Love, “Linux Kernel Development”, 3rd Edition, Addison-Wesley Professional, 2010. ISBN: 978-0672329463.

[2] The Linux Kernel documentation, “Unreliable Guide To Locking” (covers softirqs, tasklets, `spin_lock_irqsave()` and `spin_unlock_irqrestore()`). Available: `https://docs.kernel.org/kernel-hacking/locking.html`

[3] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman, “Linux Device Drivers”, 3rd Edition, O’Reilly Media, 2005. ISBN: 978-0596005900.

[4] Tejun Heo, “Concurrency Managed Workqueue (cmwq)”, The Linux Kernel documentation (Documentation/core-api/workqueue.rst). Available: `https://docs.kernel.org/core-api/workqueue.html`

[5] The Linux Kernel documentation, “Linux generic IRQ handling” (documents `request_threaded_irq()`). Available: `https://www.kernel.org/doc/htmldocs/genericirq/index.html`

[6] Marta Rybczyńska, “Modernizing the tasklet API”, LWN.net, 14 September 2020. Available: `https://lwn.net/Articles/830964`

[7] Patrick Mochel and Mike Murphy, “sysfs - *The* filesystem for exporting kernel objects”, The Linux Kernel documentation (Documentation/filesystems/sysfs.rst). Available: `https://docs.kernel.org/filesystems/sysfs.html`

[8] Linux kernel source tree, `fs/char_dev.c` (implementation of `alloc_chrdev_region()`, `cdev_add()` and `unregister_chrdev_region()`). Available: `https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/fs/char_dev.c`

[9] Bootlin, “Linux Kernel and Driver Development Training”, training slides. Available: `https://bootlin.com/doc/training/linux-kernel/linux-kernel-slides.pdf`

[10] Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, “Course Curriculum and Syllabus: 10212EC908 – Linux Device Drivers”, Department of Electronics and Communication Engineering, Academic Session 2025–2026.